## EXPERIMENT 4: - INFORMATION RETRIEVAL USING NLTK

**Learning Objective**

To preprocess text and build a basic information retrieval system using NLTK, TF-IDF, and cosine similarity.

**Problem Statement**

To retrieve relevant Amazon customer reviews based on a user's search query using text similarity.

**TASKS TO PERFORM**

1. Load the Amazon Fine Food Reviews dataset into a Pandas DataFrame.
2. Import the required Python libraries and NLTK resources.
3. Remove missing or null reviews and duplicate records.
4. Select the review text column and preprocess the text.
5. Apply preprocessing to all reviews and store them in a new column named Cleaned_Text.
6. Display the original and cleaned versions of five reviews.
7. Convert cleaned reviews into TF-IDF vectors and display the matrix shape and number of unique terms.
8. Create a retrieval function using cosine similarity to return the top five relevant reviews.
9. Test the retrieval system using the sample queries: "great product with fast shipping", "disappointed", "excellent taste", and "poor packaging". Compare the relevance of the results.

**AIM**

To develop an information retrieval system using NLTK, TF-IDF vectorization, and cosine similarity to retrieve relevant Amazon customer reviews.

**ALGORITHM**

1. Import the required libraries and load the dataset.
2. Remove missing reviews and duplicate records.
3. Clean text by removing punctuation, numbers, and stopwords.
4. Apply stemming to the cleaned reviews.

5. Convert reviews into TF-IDF vectors.
6. Calculate cosine similarity between the query and reviews.
7. Retrieve and display the top five relevant reviews for each query.
8. Calculate cosine similarity between the query and reviews, retrieve the top five matching reviews, and display their similarity scores and original text.
9. Test the retrieval system using the queries "great product with fast shipping", "disappointed", "excellent taste", and "poor packaging".
10. Compare the retrieved reviews for each query and comment on their relevance.

**PROGRAM AND OUTPUT**

```
# Question 1: Install and import the required Python libraries.
# Luei johncena (241801140)
!pip -q install nltk scikit-learn pandas
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Question 2: Load the Amazon Fine Food Reviews dataset into a Pandas DataFrame.
# Luei johncena (241801140)
url = "https://huggingface.co/datasets/PJ2005/amazon-fine-food-reviews/resolve/main/Reviews.csv"
df = pd.read_csv(url, nrows=10000)
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df.head())
```

```
Shape: (10000, 10)
Columns: ['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator', 'Score', 'Time', 'Su
```

| | Id | ProductId | UserId | ProfileName | HelpfulnessNumerator | HelpfulnessDenominator | Score | Time | Summ |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | B001E4KFG0 | A3SGXH7AUHU8GW | delmartian | 1 | 1 | 5 | 1303862400 | G Qu Dog F |

```
# Question 3: Import the required NLTK resources.
# Luei johncena (241801140)
nltk.download("stopwords", quiet=True)
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()
print("NLTK stopwords loaded:", len(stop_words))
```

```
NLTK stopwords loaded: 198
```

```
# Question 4: Select the review text column, remove missing/null reviews and duplicates, and use the first 10,000 reviews.
# Luei johncena (241801140)
df = df.dropna(subset=["Text"]).drop_duplicates().copy()
df["Text"] = df["Text"].astype(str)
df = df.head(10000).copy()
print("Rows after cleaning:", len(df))
print("Missing review texts:", df["Text"].isna().sum())
```

```
Rows after cleaning: 10000
Missing review texts: 0
```

```
# Question 5: Create a preprocessing function to lowercase text, remove punctuation/numbers/special characters, tokenize, r
# Luei johncena (241801140)
def preprocess(text):
    text = re.sub(r"<.*?>", " ", str(text).lower())
    text = re.sub(r"[^a-z\s]", " ", text)
    words = [stemmer.stem(w) for w in text.split()
             if w not in stop_words and len(w) > 1]
    return " ".join(words)

print(preprocess("Great product with fast shipping!"))
```

```
great product fast ship
```

```
# Question 6: Preprocess all reviews and store the results in a new Cleaned_Text column.
# Luei johncena (241801140)
df["Cleaned_Text"] = df["Text"].apply(preprocess)
print("Preprocessing completed for", len(df), "reviews.")
display(df[["Text", "Cleaned_Text"]].head())
```

```
Preprocessing completed for 10000 reviews.
```

| | Text | Cleaned_Text |
|---|---|---|
| 0 | I have bought several of the Vitality canned d... | bought sever vital can dog food product found ... |
| 1 | Product arrived labeled as Jumbo Salted Peanut... | product arriv label jumbo salt peanut peanut a... |
| 2 | This is a confection that has been around a fe... | confect around centuri light pillowi citru gel... |
| 3 | If you are looking for the secret ingredient i... | look secret ingredi robitussin believ found go... |
| 4 | Great taffy at a great price. There was a wid... | great taffi great price wide assort yummi taff... |

```
# Question 7: Display the original and cleaned versions of any five reviews.
# Luei johncena (241801140)
for i, row in df[["Text", "Cleaned_Text"]].head(5).iterrows():
    print("\nOriginal:", row["Text"][:400])
    print("Cleaned:", row["Cleaned_Text"][:400])
    print("-" * 50)
```

```
Original: I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The
Cleaned: bought sever vital can dog food product found good qualiti product look like stew process meat smell better labrad
--------------------------------------------------

Original: Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if thi
Cleaned: product arriv label jumbo salt peanut peanut actual small size unsalt sure error vendor intend repres product jumb
--------------------------------------------------

Original: This is a confection that has been around a few centuries.  It is a light, pillowy citrus gelatin with nuts - in
Cleaned: confect around centuri light pillowi citru gelatin nut case filbert cut tini squar liber coat powder sugar tini mo
--------------------------------------------------

Original: If you are looking for the secret ingredient in Robitussin I believe I have found it.  I got this in addition to
Cleaned: look secret ingredi robitussin believ found got addit root beer extract order good made cherri soda flavor medicin
--------------------------------------------------

Original: Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very quick.  If your a t
Cleaned: great taffi great price wide assort yummi taffi deliveri quick taffi lover deal
--------------------------------------------------
```

```
# Question 8: Convert cleaned reviews into TF-IDF vectors and display matrix shape and number of unique terms.
# Luei johncena (241801140)
vectorizer = TfidfVectorizer(max_features=5000)
tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])
print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Number of unique terms:", len(vectorizer.get_feature_names_out()))
```

```
TF-IDF matrix shape: (10000, 5000)
Number of unique terms: 5000
```

```
# Question 9: Create a retrieval function that preprocesses a query and converts it into a TF-IDF vector.
# Luei johncena (241801140)
def prepare_query(query):
    return vectorizer.transform([preprocess(query)])

print("Query preprocessing function is ready.")
```

```
Query preprocessing function is ready.
```

```
# Question 10: Calculate cosine similarity and return the top five reviews with similarity scores and original text.
# Luei johncena (241801140)
def retrieve_reviews(query, top_n=5):
    query_vector = prepare_query(query)
    scores = cosine_similarity(query_vector, tfidf_matrix).ravel()
    indices = scores.argsort()[-top_n:][::-1]
    result = df.iloc[indices][["Summary", "Text"]].copy()
    result["Similarity_Score"] = scores[indices]
    return result

display(retrieve_reviews("great product with fast shipping"))
```

| | Summary | Text | Similarity_Score |
|---|---|---|---|
| 5226 | Great product and would buy again | Enjoyed the product and they also provided ver... | 0.576027 |
| 8021 | good tea | The tea is good and fresh. We enjoy it. The sh... | 0.522053 |
| 5417 | Love these meals | My son really enjoys all these flavors- eats t... | 0.476972 |
| 5057 | Amazingly Good! | These are very good, they were a great price a... | 0.441529 |
| 6034 | energy drink | The energy drink is a great product. The shipp... | 0.424585 |

```
# Question 11: Test the retrieval system using the four sample queries.
# Luei johncena (241801140)
queries = [
    "great product with fast shipping",
    "disappointed",
    "excellent taste",
    "poor packaging"
]
for query in queries:
    print("\nQuery:", query)
    display(retrieve_reviews(query, top_n=5))
```

```
Query: great product with fast shipping
```

| | Summary | Text | Similarity_Score |
|---|---|---|---|
| 5226 | Great product and would buy again | Enjoyed the product and they also provided ver... | 0.576027 |
| 8021 | good tea | The tea is good and fresh. We enjoy it. The sh... | 0.522053 |
| 5417 | Love these meals | My son really enjoys all these flavors- eats t... | 0.476972 |
| 5057 | Amazingly Good! | These are very good, they were a great price a... | 0.441529 |
| 6034 | energy drink | The energy drink is a great product. The shipp... | 0.424585 |

```
Query: disappointed
```

| | Summary | Text | Similarity_Score |
|---|---|---|---|
| 3151 | Bread | I am a bit disappointed. The flavor was not w... | 0.524904 |
| 4295 | Better mixes out there! | When I saw the ingredients on this box I knew ... | 0.348591 |
| 4378 | good product, too expensive | The product is very good. Way too expensive an... | 0.341480 |
| 788 | disappointment | I was disappointed in this product because I t... | 0.340374 |
| 6548 | Different | Disappointed. The big boxes had a very differ... | 0.339557 |

```
Query: excellent taste
```

| | Summary | Text | Similarity_Score |
|---|---|---|---|
| 2332 | Excellent | It is very very good sweets<br />Amazon always... | 0.561229 |
| 3407 | Grove Square K-cups | We recieved the product on time and in excelle... | 0.502545 |
| 1743 | Popchips | The Popchips were excellent - had excellent fl... | 0.499514 |
| 9503 | Senseo coffee- French Vanilla Caramel | Excellent tasting. Will have to remember to o... | 0.494205 |
| 7324 | Purchased chamomile | Excellent product. Exactly what they said it w... | 0.438834 |

```
Query: poor packaging
```

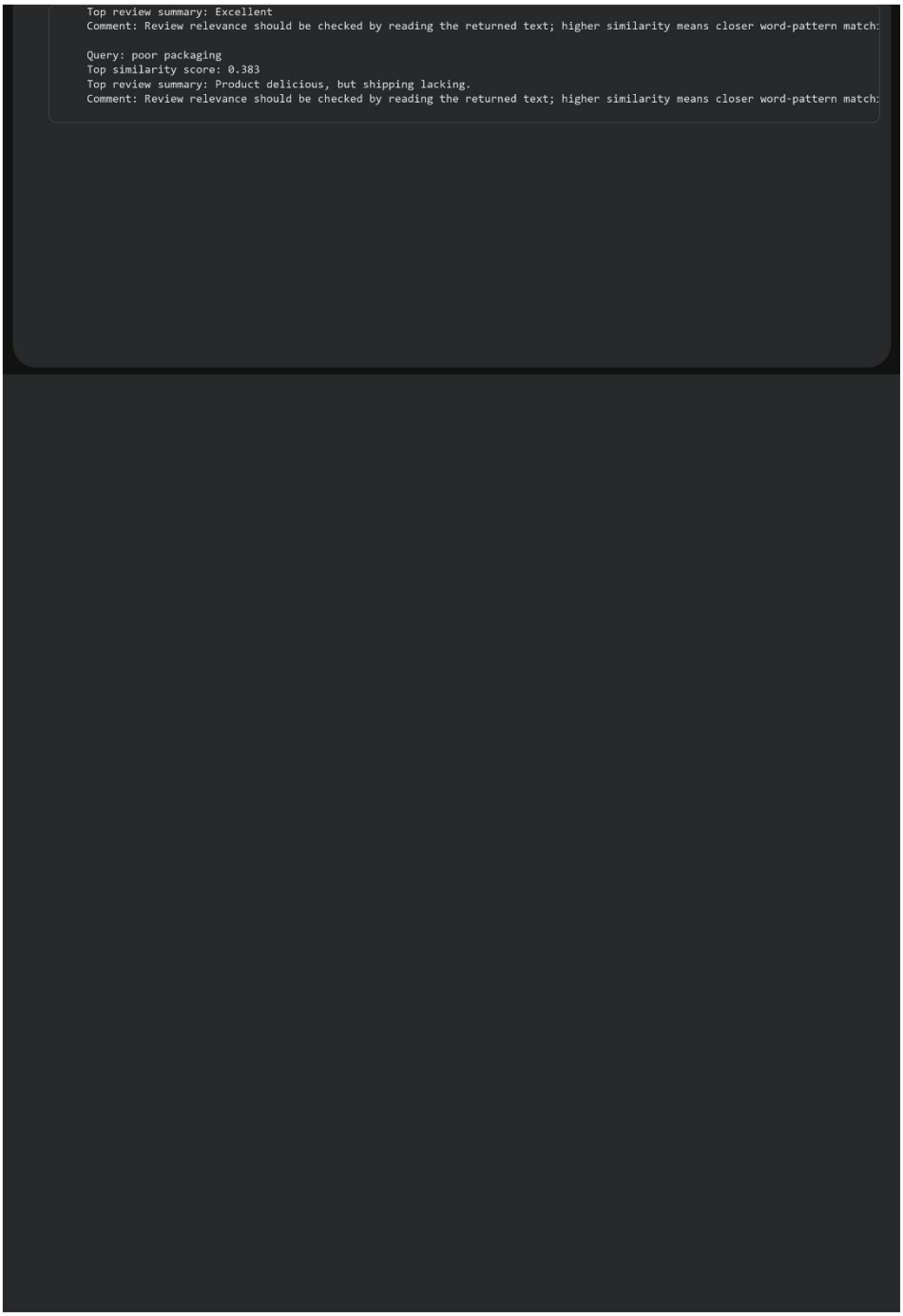
| | Summary | Text | Similarity_Score |
|---|---|---|---|
| 6185 | Product delicious, but shipping lacking. | I was delighted to find this product online, b... | 0.382790 |
| 2006 | Maybe it was just my batch? | Didnt really enjoy these after wanting to try ... | 0.329255 |
| 7579 | wonderful soup BUT.... | The soup is rich and tasty, but DON'T buy it f... | 0.323083 |
| 5032 | Very poor service | I would agree with the other poor ratings for ... | 0.293346 |
| 532 | Great price, but not as tangy as I expected. | Chips are not as tangy as I hoped or expected.... | 0.292871 |

```
# Question 12: Compare retrieved reviews for each query and comment on their relevance.
# Luei johncena (241801140)
for query in queries:
    results = retrieve_reviews(query, top_n=5)
    print("\nQuery:", query)
    print("Top similarity score:", round(results["Similarity_Score"].max(), 3))
    print("Top review summary:", results.iloc[0]["Summary"])
    print("Comment: Review relevance should be checked by reading the returned text; higher similarity means closer word-p
```

```
Query: great product with fast shipping
Top similarity score: 0.576
Top review summary: Great product and would buy again
Comment: Review relevance should be checked by reading the returned text; higher similarity means closer word-pattern match

Query: disappointed
Top similarity score: 0.525
Top review summary: Bread
Comment: Review relevance should be checked by reading the returned text; higher similarity means closer word-pattern match

Query: excellent taste
Top similarity score: 0.561
```






**RESULT**

Customer reviews were processed using NLTK to extract tokens, lemmas, POS tags, and named entities. The extracted information was displayed in a structured format.